

Spieleprogrammierung

Übungsblatt 2

Aufgabe – 3D-NPC mit KI-gesteuertem Verhalten

Aufgabe

In dieser Übung sollte das in Aufgabe 1 erstellte Labyrinth um einen nicht spielbaren Charakter (NPC), der eine gewisse Menschenähnlichkeit hat und sich entsprechend verhält. Der NPC soll als Charakter mit nachvollziehbarem Verhalten wahrgenommen werden.

Labyrinth

Für interessantere Entscheidungen und Bewegungsmöglichkeiten im Labyrinth wurde dieses etwas angepasst, im Vergleich mit der vorherigen Abgabe (siehe BodenBuilder-Skript):

```
{
  "width": 18,
  "height": 15,
  "tiles": [
    0, 0, 0, 0, 0, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3,
    0, 0, 0, 0, 3, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 3,
    0, 3, 3, 3, 3, 2, 3, 3, 2, 3, 3, 3, 3, 3, 3, 3, 2, 3,
    3, 2, 2, 2, 3, 2, 2, 2, 2, 1, 1, 1, 1, 3, 0, 3, 2, 3,
    3, 2, 3, 2, 3, 2, 2, 2, 2, 3, 1, 3, 1, 3, 0, 3, 2, 3,
    3, 2, 3, 2, 3, 2, 2, 2, 2, 3, 1, 3, 1, 3, 0, 3, 2, 3,
    3, 2, 3, 2, 3, 2, 2, 2, 2, 3, 1, 3, 1, 3, 0, 3, 2, 3,
    3, 2, 3, 2, 3, 2, 2, 2, 2, 3, 1, 3, 1, 3, 0, 3, 2, 3,
    3, 2, 3, 2, 2, 2, 2, 2, 2, 3, 1, 1, 1, 3, 0, 3, 2, 3,
    3, 2, 3, 3, 3, 3, 2, 2, 3, 0, 3, 3, 3, 3, 0, 0, 3, 2, 3,
    3, 2, 3, 0, 0, 0, 3, 2, 3, 0, 0, 0, 0, 3, 3, 3, 2, 3,
    3, 2, 3, 0, 0, 0, 3, 2, 3, 0, 0, 0, 3, 2, 2, 2, 2, 3,
    3, 2, 3, 3, 3, 3, 3, 2, 3, 3, 3, 3, 2, 2, 3, 2, 2, 3,
    3, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 3, 0, 3, 2, 3,
    0, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 0, 0, 3, 3, 0
  ]
}
```

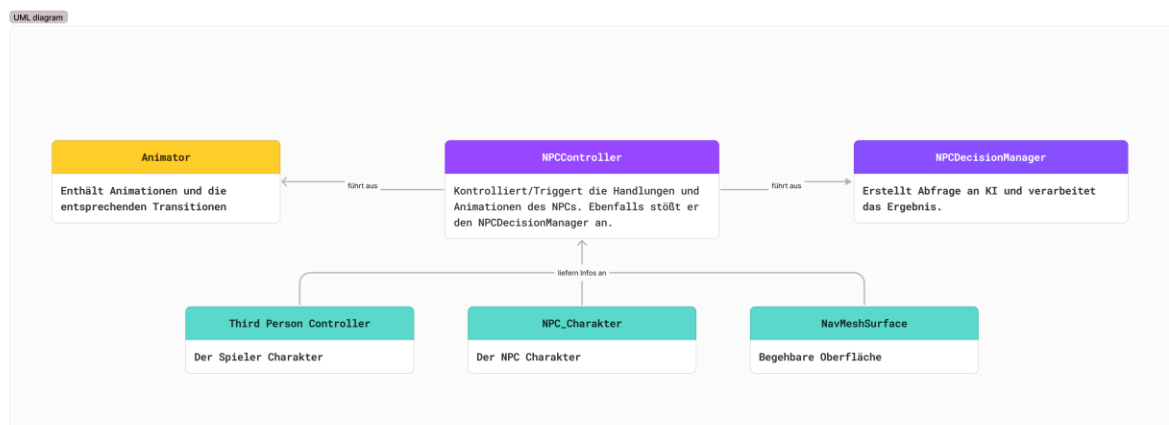
Steuerung

Der Third-Person-Charakter kann per WASD-Tasten gesteuert werden. Mit der Umschalttaste kann der Charakter laufen und mit der Maus die Richtung/Sichtfeld angepasst werden. Der NPC beherrscht folgende Bewegungsmöglichkeiten:

switch (action)

```
{  
    case "GehZuSpieler":  
        MoveTowards(player.position, speed * 0.5f);  
        break;  
    case "RennWeg":  
        MoveAwayFrom(player.position, speed * 1.0f);  
        break;  
    case "KriecheZuSpieler":  
        CrouchTo(player.position, speed * 0.2f);  
        break;  
    case "KriecheWeg":  
        CrouchAwayFrom(player.position, speed * 0.2f);  
        break;  
    case "LaufeZuSpieler":  
        RunTowards(player.position, speed * 1f);  
        break;  
    case "LaufeWeg":  
        RunAwayFrom(player.position, speed * 1f);  
        break;  
    case "WinkeDemSpieler":  
        WaveTowards(player.position, 0);  
        break;  
}
```

Technische Umsetzung



Komponentenbeschreibung:

- **NPCController**
Zentrale Steuereinheit des NPCs. Dieses Skript ist verantwortlich für die Handlungslogik und das Triggern der entsprechenden Animationen. Es ruft regelmäßig den NPCDecisionManager auf, um situativ passende Entscheidungen einzuholen. Der NPCController analysiert außerdem die Spielumgebung und leitet daraus Aktionsentscheidungen ab.
- **NPCDecisionManager**
Zuständig für die Kommunikation mit dem lokalen KI-Modell (z. B. Mistral via Ollama). Er erzeugt einen Prompt aus der aktuellen Spielsituation, sendet diesen an die KI und verarbeitet die Antwort zur Aktionsbestimmung.
- **Animator**
Verwaltet die Animationen des NPCs. Der NPCController aktiviert und wechselt Animationen über diesen Animator (z. B. durch Setzen von Triggern oder Parametern), abhängig von der gewählten Aktion.
- **Third Person Controller**
Der spielergesteuerte Charakter. Dessen Position, Blickrichtung und Bewegungsverhalten dienen als Input für die Situationsanalyse des NPCs.
- **NPC_Charakter**
Das konkrete NPC-Objekt im Spiel, auf das sich der Controller bezieht. Enthält Komponenten wie NavMeshAgent, Animator und Collider, und stellt die physische und visuelle Repräsentation des NPCs dar.
- **NavMeshSurface**
Beschreibt die begehbare Fläche des Labyrinths. Sie ermöglicht dem NPC mithilfe des NavMeshAgents, sich intelligent in der Umgebung zu bewegen (z. B. beim Weglaufen oder Verfolgen des Spielers).

Zustandserfassung / Situationserkennung des NPC

Der NPC analysiert regelmäßig die aktuelle Spielsituation durch folgende Parameter:

- Abstand zum Spieler
→ `Vector3.Distance(transform.position, player.position)`
- Blickrichtung beider Charaktere
→ Berechnet über `Vector3.Angle(...)` zwischen Blick- und Positionsvektor
- Bewegungsrichtung und -geschwindigkeit des Spielers
→ Aus der Differenz von aktueller und vorheriger Position

Die Umsetzung und das Triggern des KI-Modells findet in dem Skript `NPCController` statt.

Kommunikation mit dem KI-Modell

Als KI habe ich Mistral verwendet, welches ich mit Hilfe von Ollama lokal auf meinem Rechner laufen ließ. Die ermittelten Werte werden in einem sprachlichen Prompt zusammengefasst und über eine HTTP-POST-Anfrage an das Modell weitergeleitet. Die Umsetzung der Kommunikation findet in dem Skript `NPCDecisionManager` statt.

API-Endpunkt: `http://localhost:11434/api/generate`

Prompt-Beispiel:

„Die Person ist 3.4 Meter entfernt. Sie läuft seitlich. Schaut die Person dich an? Ja. Schaut du ihn an? Nein. Gib genau eine der folgenden Aktionen zurück: [...]“

Die Anfrage wird in Unity per `UnityWebRequest` abgesendet und als JSON-Response ausgewertet.

Neben dem Modell Mistral habe ich noch folgende getestet:

1. DeepSeek-Coder
Hier bekam ich keine verwertbaren Antworten von dem Modell, da die Anfragen nicht technisch genug waren.
2. Llama3
Eine KI von Meta, die über Sprachverständlichkeit verfügt und robuster ist bei einfachen Entscheidungsaufforderungen.
3. Phi-3
Eine KI von Microsoft, die leichtgewichtig und schnell ist.

Ausführung der Aktion

Die vom Modell zurückgegebene Aktion wird über einen switch-Block umgesetzt. Jede Aktion ruft gezielt ein Bewegungs- oder Animationsverhalten auf:

Aktion	Verhalten
GehZuSpieler	NPC nähert sich langsam
KriecheWeg	NPC kriecht weg
WinkeDemSpieler	NPC winkt animiert

Ungültige Antworten werden verworfen – nur **gültige Schlüsselwörter** führen zu Aktionen. Die Positionen zum Hin- oder Weglaufen vom Spieler werden durch die Position des Spielers (zum Hinlaufen) und einer Methode, zum Finden der am weitesten entfernten Position vor dem Spieler generiert.

Ergebnisbewertung

Die von dem Modell erhaltenen Antworten sind leider oft gleich und nicht abwechslungsreich. Verschiedene KI-Modelle bevorzugen oft das Wiederholen einer Antwort, weshalb ein tatsächliches menschenähnliches Verhalten nicht wirklich erreicht wurde. Zusätzlich gab es - vermutlich aufgrund von nicht ausreichender Hardware - öfters Probleme mit den Antworten der KI. Bei den verschiedenen KIs konnten kaum Abstufungen in den Antworten erhalten werden.